

LABORATORIO 10 - Arquitectura de Software

TEAM:

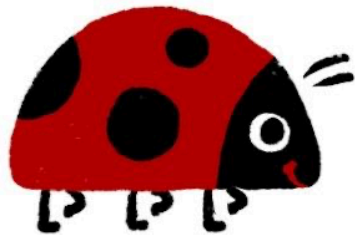
Proyecto: Bot - MonitorMach

Objetivos:

- Diseñar una arquitectura basada en microservicios.
- Implementar logging distribuido.
- Medir latencia y disponibilidad.
- Procesar logs para generar métricas.
- Ejecutar pruebas de carga.
- Construir comandos tipo bot/CLI.

El bot no tiene ningún componente de AI solo interpreta comandos pre-definidos.

Bot - Monitor Mach



Comandos:

CheckLatency

`<module> <start-date> <end-date>`

CheckAvailability

`<module> -[Last5Days, Last7Days]`

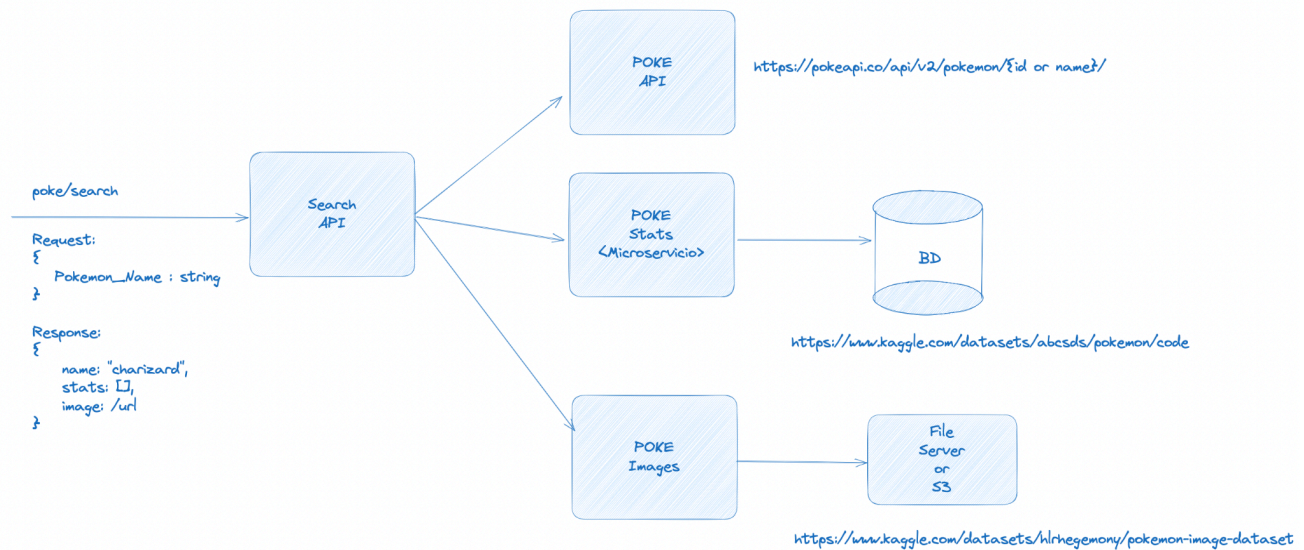
RenderGraph `-[Availability, Latency]`

`<module> -[Last5Days, Last7Days]`

Parte I (10 ptos)

- Implementar el siguiente diseño:

(*) Lenguaje libre (Solo se debe implementar backend)



- (*) Cada Microservicio/Modulo debe tener su propio manejo de logs.
- (**) Los logs deben tener una sintaxis o nomenclatura estandar para todos los microservicios. {Fecha}{Modulo}{API}{Funcion} Message
- (**) Los logs deben identificar claramente tiempo y a que microservicio pertenecen.

(**) Para medir la latencia asumir que debe medir el tiempo de inicio y fin de cada bloque de codigo.

URLS de Fuente de Datos:

POKE_API: <https://pokeapi.co/api/v2/pokemon/{id or name}/>

POKE_STATS: <https://www.kaggle.com/datasets/abcsds/pokemon/code>

POKE_IMG:

<https://www.kaggle.com/datasets/hlrhegemony/pokemon-image-dataset>

Entregable:

Repo:

JMeter(jmx) o Locust:

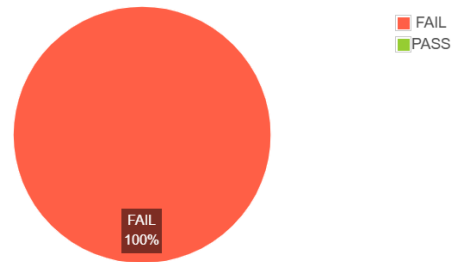
(*) Este jmeter debe tener un minimo de 1000 llamadas y un maximo de 10000 llamadas. Esto es necesario para generar los logs.

Screenshot de Response:

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.000	500 ms	1 sec 500 ms	Total
0.000	500 ms	1 sec 500 ms	GET /poke/search

Requests Summary



Statistics

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	10000	10000	100.00%	16750.37	14950	20277	15695.00	20003.00	20004.00	20012.99	5.59	4.14	0.63
GET /poke/search	10000	10000	100.00%	16750.37	14950	20277	15695.00	20003.00	20004.00	20012.99	5.59	4.14	0.63

Parte II (10 ptos)

- Implementar comandos del Bot.
 - **CheckLatency**: Muestra la latencia de la aplicación de un modulo en un periodo determinado.
 - **CheckAvailaibilty**: Muestra la disponibilidad del servicio durante X días.
 - **Availability** = $\text{Tasa de Éxito} / (\text{Tasa de Éxito} + \text{Tasa de Error})$
 - Tasa de Éxito** = Número de Requests con HTTP Code 200.
 - Tasa de Fallo** = Número de Requests con HTTP Code 500.
 - **RenderGraph**: Renderiza una grafica lineal con las tendencias de disponibilidad/latencia para un modulo en un periodo de X días. (*) El diagrama puede ser en consola. ASCII ART:

```
<=====] }=====
      --. /|
      \"/_.' /
      .'_.' /
      :/_\{/ /
      (L /--' /-----
      | /-\ .'-'\ / |
      : /-\ .'-'\ / |
```

$$\begin{array}{ccccc} \text{snd} & \backslash \backslash , & | | & & \backslash | \\ & \backslash / & | | & & | | \end{array}$$

- **Stats:** Muestra metricas mas generales:
 - P95 latency
 - requests por minuto
 - error ratio
 - throughput
 - top failing endpoint

(*) La latencia corresponde al tiempo total de respuesta de cada request HTTP por microservicio.

(*) Toda esta informacion se obtendra de los logs previamente desarrollados.

Use Cases:

CheckAvailability PokeStats -Last5Days

01/10	99.9%
02/10	89.9%
03/10	89.8%
04/10	90.5%
05/10	87.8%

CheckLatency **PokeImage** -01/10 -03/10

01/10 3000ms
02/10 4000ms
03/10 1500ms

RenderGraph -Latency -Last3Days

		4000
3000		
		1500
01/10	02/10	03/10

Entregables:

Repo: <código de bot>

Incluir imágenes en el repo de los resultados de la ejecución de los comandos de su bot

```
PS C:\Users\HP\Desktop\UTEC\ArquiSoft\semana10\app\bot> python bot.py --logs-dir ../logs CheckAvailability PokeStats -Last5D
```

```
[CheckAvailability] POKE_STATS
```

```
-----
```

```
Disponibilidad = Exitos / (Exitos + Errores) x 100
```

```
29/05 sin datos
```

```
30/05 sin datos
```

```
31/05 sin datos
```

```
01/06 sin datos
```

```
02/06 100.0% (2395 ok / 0 err)
```

```
Disponibilidad total : 100.0%
```

```
PS C:\Users\HP\Desktop\UTEC\ArquiSoft\semana10\app\bot> python bot.py --logs-dir ../logs CheckLatency PokeStats -Last3Days
```

```
[CheckLatency] POKE_STATS
```

```
-----
```

```
31/05 sin datos
```

```
01/06 sin datos
```

```
02/06 19 ms (n=2395)
```

```
Promedio total : 19 ms
```

```
P95 : 39 ms
```

```
PS C:\Users\HP\Desktop\UTEC\ArquiSoft\semana10\app\bot> python bot.py --logs-dir ../logs RenderGraph -Latency -Last3Days PokeStats
```

```
[RenderGraph] Latencia - Ultimos 3 días
```

```
Latencia - POKE_STATS
```

```
19ms |
```

```
19ms |
```

```
19ms |
```

```
19ms |
```

```
19ms |
```

```
19ms | **19ms**
```

```
19ms |
```

```
19ms |
```

```
19ms |
```

```
19ms |
```

```
+-----
```

```
02/06
```

```
PS C:\Users\HP\Desktop\UTEC\ArquiSoft\semana10\app\bot> python bot.py --logs-dir ../logs Stats
```

```
[Stats] Stats | TODOS | Ultimos 1 dia(s)
```

```
-----
```

Total requests	7143
Requests / minuto	58.27
Throughput	0.9711 req/s
Error ratio	0.0%
Latencia promedio	3441 ms
P95 latencia	10822 ms
Top failing endpoint	ninguno (0.0% error rate)

Latencia por modulo:

POKE_API	avg=	555ms	p95=	565ms	errors=0.0%
POKE_IMAGES	avg=	35ms	p95=	80ms	errors=0.0%
POKE_STATS	avg=	19ms	p95=	39ms	errors=0.0%
SEARCH_API	avg=	10448ms	p95=	12154ms	errors=0.0%

Analisis:

[Bottleneck] **SEARCH_API**

P95 = 12154 ms – orquestacion sincronica de 3 servicios en paralelo

[Retry recomendado]

* **Ninguno** – error ratio < 2% en todos los modulos

[Debe escalar?]

* **Si** –
- 58.3 req/min > umbral 50
- P95 10822 ms > 3000 ms